Assignment – 3

ID5130: Parallel Scientific Computing

Indian Institute of Technology Madras

Shubham Jadhav

Roll No: CS24S009

Date: April 2026

Contents

1 Hand Calculation 1: Speedup, Efficiency, and Amdahl's Law **2**
1.1 Given Data . . . 2
1.2 Step 1: Compute Sequential and Parallel Parts . . . 2
1.3 Step 2: Compute Communication Overhead . . . 2
1.4 Step 3: Parallel Execution Time . . . 2
1.5 Part (a): Speedup $\psi(n, p)$ . . . 3
1.6 Part (b): Efficiency ϵ . . . 3
1.7 Part (c): Sequential Fraction of the Program . . . 3
1.8 Part (d): Amdahl's Law Speedup with $p = 100$ . . . 3
1.9 Part (e): Maximum Speedup (Amdahl's Law, $p \to \infty$) . . . 4

2 Hand Calculation 2: Recursive Doubling for Tridiagonal System **5**
2.1 Problem Statement . . . 5
2.2 Recursive Doubling (Cyclic Reduction) Algorithm . . . 5
2.3 Level 1: Eliminate x_1 and x_3 (Odd-Indexed Unknowns) . . . 5
2.4 Level 2: Reduce to Single Equation for x_4 . . . 6
2.5 Forward Solve . . . 7
2.6 Back-Substitution . . . 7
2.7 Solution and Verification . . . 7
2.8 Summary of Recursive Doubling Steps . . . 7

3 Programming Question: Cholesky Decomposition with OpenACC **8**
3.1 Algorithm . . . 8
3.2 Bug Fix in Given Code . . . 8
3.3 Dependency Analysis and Choice of Parallelization Axis . . . 8
3.4 OpenACC Parallelization Strategy . . . 9
3.4.1 Data Transfer Optimization . . . 9
3.4.2 Parallelism . . . 9
3.4.3 Gang and Vector Length Tuning . . . 9
3.4.4 V2: Blocked / Tiled Cholesky (Optimized) . . . 10
3.4.5 Correctness . . . 10
3.5 Terminal Output and Verification (N = 10) . . . 10
3.6 Timing Results . . . 11
3.7 Plot: Serial vs OpenACC Timing . . . 12
3.8 Analysis of Results . . . 12

4 Reference Material Used **13**

Hand Calculation 1: Speedup, Efficiency, and Amdahl's Law

Given Data

$$\begin{aligned}
\sigma(n) &= an + b \quad \text{(sequential part)} \\
\phi(n) &= cn^3 + dn^2 + en \quad \text{(parallelizable part)} \\
a &= 10, \quad b = 1000 \\
c &= 10^{-3}, \quad d = 0.2, \quad e = 100 \\
n &= 1000, \quad p = 100 \\
\text{Communication locations} &= \lceil \log_{10} n \rceil \\
\text{Cost per communication} &= 1000 \log_{10} p + \frac{n}{10}
\end{aligned}$$

Step 1: Compute Sequential and Parallel Parts

Sequential part $\sigma(1000)$:

$$\sigma(1000) = 10 \times 1000 + 1000 = 10{,}000 + 1{,}000 = 11{,}000 \text{ units} \tag{1}$$

Parallelizable part $\phi(1000)$:

$$\begin{aligned}
\phi(1000) &= 10^{-3} \times 1000^3 + 0.2 \times 1000^2 + 100 \times 1000 \\
&= 10^{-3} \times 10^9 + 0.2 \times 10^6 + 10^5 \\
&= 1{,}000{,}000 + 200{,}000 + 100{,}000 = 1{,}300{,}000 \text{ units}
\end{aligned} \tag{2}$$

Total serial execution time $T(n, 1)$:

$$T(1000, 1) = \sigma(1000) + \phi(1000) = 11{,}000 + 1{,}300{,}000 = 1{,}311{,}000 \text{ units} \tag{3}$$

Step 2: Compute Communication Overhead

Number of communication locations:

$$\lceil \log_{10}(1000) \rceil = \lceil 3.0 \rceil = 3 \tag{4}$$

Cost per communication event:

$$1000 \cdot (\log_{10} 100) + \frac{1000}{10} = 1000 \times 2 + 100 = 2{,}100 \text{ units} \tag{5}$$

Total communication overhead:

$$T_{\text{comm}} = 3 \times 2{,}100 = 6{,}300 \text{ units} \tag{6}$$

Step 3: Parallel Execution Time

The parallel execution time on p processes is:

$$T(n, p) = \sigma(n) + \frac{\phi(n)}{p} + T_{\text{comm}} \tag{7}$$

$$\begin{aligned}
T(1000, 100) &= 11{,}000 + \frac{1{,}300{,}000}{100} + 6{,}300 \\
&= 11{,}000 + 13{,}000 + 6{,}300 = \mathbf{30{,}300} \text{ units}
\end{aligned} \tag{8}$$

Part (a): Speedup $\psi(n,p)$

$$\boxed{\psi(n,p) = \frac{T(n,1)}{T(n,p)} = \frac{1{,}311{,}000}{30{,}300} \approx 43.27} \tag{9}$$

Part (b): Efficiency ϵ

$$\boxed{\epsilon = \frac{\psi(n,p)}{p} = \frac{43.27}{100} \approx 0.4327 = 43.27\%} \tag{10}$$

The efficiency of 43.3% means that on average each process is productively used 43.3% of the time; the remaining 56.7% is lost to synchronization, communication overhead, and the sequential portion.

Part (c): Sequential Fraction of the Program

The sequential fraction f is defined as the fraction of the total serial work that is inherently sequential:

$$f = \frac{\sigma(n)}{\sigma(n) + \phi(n)} = \frac{11{,}000}{1{,}311{,}000} = \frac{11}{1311} \approx 0.008391 \tag{11}$$

$$\boxed{f \approx 0.8391\%} \tag{12}$$

This means approximately 0.84% of the total work cannot be parallelized.

Part (d): Amdahl's Law Speedup with $p = 100$

Amdahl's law predicts speedup as:

$$\psi_{\text{Amdahl}}(p) = \frac{1}{f + \dfrac{1-f}{p}} \tag{13}$$

Substituting $f = 11/1311$ and $p = 100$:

$$\begin{aligned}
\psi_{\text{Amdahl}}(100) &= \frac{1}{\dfrac{11}{1311} + \dfrac{1 - \frac{11}{1311}}{100}} \\
&= \frac{1}{\dfrac{11}{1311} + \dfrac{\frac{1300}{1311}}{100}} \\
&= \frac{1}{\dfrac{11}{1311} + \dfrac{1300}{131100}} \\
&= \frac{1}{\dfrac{1100}{131100} + \dfrac{1300}{131100}} \\
&= \frac{1}{\dfrac{2400}{131100}} = \frac{131100}{2400}
\end{aligned} \tag{14}$$

$$\boxed{\psi_{\text{Amdahl}}(100) = \frac{131100}{2400} = 54.625} \tag{15}$$

Comparison: The actual speedup $\psi = 43.27$ is lower than Amdahl's prediction of 54.625. This is because Amdahl's law ignores communication overhead ($T_{\text{comm}} = 6300$ units), which accounts for the difference. The actual parallel overhead is higher than the pure sequential fraction captured by Amdahl's law.

Part (e): Maximum Speedup (Amdahl's Law, $p \to \infty$)

As $p \to \infty$, the $\frac{1-f}{p}$ term vanishes:

$$\psi_{\max} = \lim_{p\to\infty} \frac{1}{f + \frac{1-f}{p}} = \frac{1}{f} = \frac{1311}{11} \tag{16}$$

$$\boxed{\psi_{\max} = \frac{1311}{11} \approx 119.18} \tag{17}$$

This is the theoretical maximum speedup: no matter how many processors are used, the sequential fraction $f \approx 0.84\%$ of the program limits speedup to at most about 119.18x. This illustrates Amdahl's law's key insight, even a small sequential fraction severely limits scalability.

Table 1: Summary of Hand Calculation 1

Quantity	**Formula**	**Value**
$T(n,1)$	$\sigma + \phi$	1,311,000 units
T_{comm}	3×2100	6,300 units
$T(n,p)$	$\sigma + \phi/p + T_{\text{comm}}$	30,300 units
$\psi(n,p)$	$T(n,1)/T(n,p)$	43.27
ϵ	ψ/p	43.27%
f	$\sigma/(\sigma+\phi)$	0.8391%
$\psi_{\text{Amdahl}}(100)$	$1/(f + (1-f)/100)$	54.625
$\psi_{\max}$	$1/f$	119.18

Hand Calculation 2: Recursive Doubling for Tridiagonal System

Problem Statement

Solve the tridiagonal system:

$$3x_1 - x_2 = 2 \tag{18}$$
$$-x_1 + 3x_2 - x_3 = 1 \tag{19}$$
$$-x_2 + 3x_3 - x_4 = 1 \tag{20}$$
$$-x_3 + 3x_4 = 2 \tag{21}$$

In matrix form $Ax = d$:

$$\begin{pmatrix} 3 & -1 & 0 & 0 \\ -1 & 3 & -1 & 0 \\ 0 & -1 & 3 & -1 \\ 0 & 0 & -1 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 1 \\ 2 \end{pmatrix} \tag{22}$$

With notation: a_i (subdiagonal) $= -1$, b_i (diagonal) $= 3$, c_i (superdiagonal) $= -1$, d_i (RHS).

Recursive Doubling (Cyclic Reduction) Algorithm

Philosophy: In each round (level), eliminate the *odd-indexed* unknowns by combining each even-indexed equation with its odd-indexed neighbors. After each round, the system size halves. With $N = 4$ equations, we need $\log_2 4 = 2$ rounds to reduce to a single equation, followed by back-substitution.

General update formulas for level s (eliminating equation $i - 2^s$ and $i + 2^s$ from equation i):

Let $\alpha_i = -a_i/b_{i-2^s}$ (left multiplier) and $\gamma_i = -c_i/b_{i+2^s}$ (right multiplier). Then:

$$a'_i = \alpha_i \cdot a_{i-2^s} \tag{23}$$
$$b'_i = b_i + \alpha_i \cdot c_{i-2^s} + \gamma_i \cdot a_{i+2^s} \tag{24}$$
$$c'_i = \gamma_i \cdot c_{i+2^s} \tag{25}$$
$$d'_i = d_i + \alpha_i \cdot d_{i-2^s} + \gamma_i \cdot d_{i+2^s} \tag{26}$$

Level 1: Eliminate x_1 and x_3 (Odd-Indexed Unknowns)

Step 1.1: Update Equation 2 (eliminate x_1 using Eq. 1 and x_3 using Eq. 3)

Left multiplier (from Eq. 1):

$$\alpha_2 = -\frac{a_2}{b_1} = -\frac{-1}{3} = +\frac{1}{3}$$

Right multiplier (from Eq. 3):

$$\gamma_2 = -\frac{c_2}{b_3} = -\frac{-1}{3} = +\frac{1}{3}$$

Updated coefficients for Equation 2:

$$b_2' = b_2 + \alpha_2 c_1 + \gamma_2 a_3 = 3 + \frac{1}{3}(-1) + \frac{1}{3}(-1) = 3 - \frac{1}{3} - \frac{1}{3} = \frac{7}{3}$$
$$a_2' = \alpha_2 \cdot a_1 = \frac{1}{3} \times 0 = 0 \quad \text{(Eq. 1 has no } x_0 \text{ term)}$$
$$c_2' = \gamma_2 \cdot c_3 = \frac{1}{3} \times (-1) = -\frac{1}{3}$$
$$d_2' = d_2 + \alpha_2 d_1 + \gamma_2 d_3 = 1 + \frac{1}{3}(2) + \frac{1}{3}(1) = 1 + \frac{2}{3} + \frac{1}{3} = 2$$

Equation 2 after Level 1:

$$\frac{7}{3}x_2 - \frac{1}{3}x_4 = 2 \tag{27}$$

Step 1.2: Update Equation 4 (eliminate x_3 using Eq. 3; no right neighbor)
Left multiplier (from Eq. 3):

$$\alpha_4 = -\frac{a_4}{b_3} = -\frac{-1}{3} = +\frac{1}{3}$$

Right multiplier: $\gamma_4 = 0$ (no Equation 5).
Updated coefficients for Equation 4:

$$b_4' = b_4 + \alpha_4 c_3 = 3 + \frac{1}{3}(-1) = 3 - \frac{1}{3} = \frac{8}{3}$$
$$a_4' = \alpha_4 \cdot a_3 = \frac{1}{3} \times (-1) = -\frac{1}{3}$$
$$c_4' = 0$$
$$d_4' = d_4 + \alpha_4 d_3 = 2 + \frac{1}{3}(1) = \frac{7}{3}$$

Equation 4 after Level 1:

$$-\frac{1}{3}x_2 + \frac{8}{3}x_4 = \frac{7}{3} \tag{28}$$

After Level 1: We have a 2×2 system for the even-indexed unknowns $\{x_2, x_4\}$:

$$\frac{7}{3}x_2 - \frac{1}{3}x_4 = 2 \tag{2'}$$
$$-\frac{1}{3}x_2 + \frac{8}{3}x_4 = \frac{7}{3} \tag{4'}$$

Level 2: Reduce to Single Equation for x_4

Eliminate x_2 from Equation (4') using Equation (2').
Left multiplier:

$$\alpha_4' = -\frac{a_4'}{b_2'} = -\frac{-1/3}{7/3} = \frac{1/3}{7/3} = \frac{1}{7}$$

Updated coefficients for Equation (4'):

$$b_4'' = b_4' + \alpha_4' \cdot c_2' = \frac{8}{3} + \frac{1}{7} \times \left(-\frac{1}{3}\right) = \frac{8}{3} - \frac{1}{21} = \frac{56}{21} - \frac{1}{21} = \frac{55}{21}$$
$$d_4'' = d_4' + \alpha_4' \cdot d_2' = \frac{7}{3} + \frac{1}{7} \times 2 = \frac{7}{3} + \frac{2}{7} = \frac{49}{21} + \frac{6}{21} = \frac{55}{21}$$

The reduced Equation (4"):

$$\frac{55}{21}x_4 = \frac{55}{21} \tag{29}$$

Forward Solve

Solve for x_4:

$$\boxed{x_4 = \frac{55/21}{55/21} = 1} \tag{30}$$

Back-Substitution

Solve for x_2 from Equation (2'): $\frac{7}{3}x_2 - \frac{1}{3}x_4 = 2$

$$\begin{aligned} \frac{7}{3}x_2 - \frac{1}{3}(1) &= 2 \\ \frac{7}{3}x_2 &= 2 + \frac{1}{3} = \frac{7}{3} \\ \boxed{x_2} &= \boxed{1} \end{aligned} \tag{31}$$

Solve for x_1 from original Equation (18): $3x_1 - x_2 = 2$

$$\begin{aligned} 3x_1 - 1 &= 2 \\ \boxed{x_1} &= \boxed{1} \end{aligned} \tag{32}$$

Solve for x_3 from original Equation (20): $-x_2 + 3x_3 - x_4 = 1$

$$\begin{aligned} -1 + 3x_3 - 1 &= 1 \\ 3x_3 &= 3 \\ \boxed{x_3} &= \boxed{1} \end{aligned} \tag{33}$$

Solution and Verification

$$\boxed{x_1 = x_2 = x_3 = x_4 = 1} \tag{34}$$

Verification:

$$\begin{aligned} 3(1) - (1) &= 2\checkmark \\ -(1) + 3(1) - (1) &= 1\checkmark \\ -(1) + 3(1) - (1) &= 1\checkmark \\ -(1) + 3(1) &= 2\checkmark \end{aligned}$$

Summary of Recursive Doubling Steps

Table 2: Recursive Doubling: System reduction summary

Level	Operation	System Size	Unknowns Remaining
0	Original system	4×4	$\{x_1, x_2, x_3, x_4\}$
1	Eliminate x_1, x_3	2×2	$\{x_2, x_4\}$
2	Eliminate x_2	1×1	$\{x_4\}$
–	Forward solve	—	$x_4 = 1$
–	Back-sub (Level 1)	—	$x_2 = 1$
–	Back-sub (Level 0)	—	$x_1 = 1,\ x_3 = 1$

The recursive doubling algorithm reduces an N-equation tridiagonal system in $\log_2 N$ reduction rounds plus $\log_2 N$ back-substitution rounds, giving a total of $\mathcal{O}(\log_2 N)$ parallel steps (versus $\mathcal{O}(N)$ for sequential Gaussian elimination).

Programming Question: Cholesky Decomposition with OpenACC

Algorithm

Given a symmetric positive-definite matrix $A[N][N]$, Cholesky decomposition finds a lower-triangular L such that $A = LL^T$. The element-wise formulas are:

$$L(i,j) = \begin{cases} \sqrt{A(i,i) - \sum_{k=0}^{i-1} L(i,k)^2} & i = j \\ \dfrac{A(i,j) - \sum_{k=0}^{j-1} L(i,k)\, L(j,k)}{L(j,j)} & i > j \end{cases} \tag{35}$$

The computation is performed *in-place*: A is overwritten with L.

Bug Fix in Given Code

The given serial code calls `init(a)` on line 1 of `main()`, but the function defined in the file is named `initmult(a)`. This causes a compilation error. The fix is to replace `init(a)` with `initmult(a)`.

Dependency Analysis and Choice of Parallelization Axis

The original row-oriented formulation in the given code has three nested loops (ii, jj, kk) where only the innermost `kk` reduction is parallelizable. Because the reduction has length at most $jj < N$, each parallel kernel does very little work, and the kernel must be launched for every (ii, jj) pair, giving $\mathcal{O}(N^2/2)$ kernel launches. For $N = 1000$, that is **500,500 kernel launches**, each with trivial work, so GPU launch overhead dominates.

The algorithm is therefore restructured to a **column-oriented Cholesky**, which exposes much more parallelism per kernel. For each column jj (processed sequentially from left to right):

Step A: Compute the diagonal element

$$L(jj,jj) = \sqrt{A(jj,jj) - \textstyle\sum_{kk<jj} L(jj,kk)^2}.$$

This is a scalar reduction, done once per column.

Step B: Compute *all* below-diagonal elements in that column,

$$L(ii,jj) = \frac{1}{L(jj,jj)} \left(A(ii,jj) - \sum_{kk=0}^{jj-1} L(ii,kk)\, L(jj,kk) \right), \quad ii = jj+1, \ldots, N-1,$$

in parallel. All of these $L(ii, jj)$ values are independent of each other (they depend only on columns $0, \ldots, jj-1$, which are already finalized).

This reduces the kernel count from $\mathcal{O}(N^2/2)$ to $2N$: only ∼2000 launches for $N = 1000$ (a 250× reduction). Step B contains $(N - jj - 1)$ independent threads each doing a dot product of length jj, which provides enough work per kernel to saturate a GPU.
Both formulations produce the same L matrix because Cholesky decomposition is unique for a positive-definite matrix. Numerical verification (see Section 3.6) confirms this.

OpenACC Parallelization Strategy

Data Transfer Optimization

The matrix is transferred to device **once** at entry and copied back **once** at exit of a single `acc data` region:

```
#pragma acc data copy(a[0:n*n]) { ... }
```

Between Step A and Step B, only the single diagonal scalar $L(jj, jj)$ is pushed to device:

```
#pragma acc update device(a[jj*n + jj : 1])
```

All Step B updates occur on the device; no per-element host-device traffic.

Parallelism

The main parallel kernel (Step B) is:

```
#pragma acc parallel loop gang vector present(a[0:n*n])\
    num_gangs(ngangs) vector_length(128) private(kk)
for (ii = jj + 1; ii < n; ++ii) {
    TYPE local_s = 0.0f;
#pragma acc loop seq
    for (kk = 0; kk < jj; ++kk)
        local_s += a[ii*n + kk] * a[jj*n + kk];
    a[ii*n + jj] = (a[ii*n + jj] - local_s) / diag;
}
```

`gang vector` distributes the outer ii loop across GPU gangs (thread blocks) and vector lanes. The inner kk loop is marked `seq` so each thread accumulates its own partial sum without inter-thread reduction, optimal for this access pattern.

Gang and Vector Length Tuning

`num_gangs` is adaptive based on N:

- $N < 256$: `num_gangs = 32` (small matrices don't need many gangs and over-subscription hurts).
- $N \geq 256$: `num_gangs = 256` (sufficient parallelism; on NVIDIA GPUs this matches typical SM count).

`vector_length(128)` gives 128 threads per gang = 4 warps on NVIDIA hardware, a standard efficient choice.

V2: Blocked / Tiled Cholesky (Optimized)

V2 restructures the algorithm into three GPU phases per block column of width $NB = 64$:

1. **POTRF** — factor the $NB \times NB$ diagonal block serially on device (`#pragma acc serial`).
2. **TRSM** — for each row $i > jb+NB$, solve $L[i, jb{:}jb+NB]$ with `gang`-parallel rows; inner j-loop is sequential (column dependency within block).
3. **SYRK** — update trailing submatrix $A[i,j] -= \sum_k L[i, jb+k] \cdot L[j, jb+k]$ with `collapse(2)` gang-vector parallelism.

Key improvements: only $3 \times \lceil N/NB \rceil$ kernel launches (vs. $2N$ for V1), SYRK is GEMM-like with $O((N - jb)^2 \times NB)$ FLOPs and massive independent thread count, and all computation stays on device.

Correctness

- Step A (POTRF) completes before TRSM begins, enforced by the implicit barrier between consecutive `acc parallel` regions.
- Within TRSM, every gang reads only finalized columns $[0, jb)$ and its own block column; no race conditions.
- Within SYRK, each (i, j) update is independent (lower triangle, gang-vector parallel).
- Numerical verification for $N = 10$ and $N = 1000$: both V1 and V2 produce results within floating-point tolerance of serial (max difference $< 10^{-7}$). Result: **PASS**.

Terminal Output and Verification (N = 10)

Compiled with `nvc -acc -gpu=cc50 -O3` and executed on the MX130 GPU (device type = 4, NVIDIA, cc50):

```
============================================================
  Cholesky: Serial vs OpenACC V1 (Column) vs V2 (Blocked)
============================================================
  OpenACC enabled. Device type: 4
  Number of devices: 1
  Block size NB = 64

------------------------------------------------------------
  N = 10  (10 x 10 matrix, 0.0 MB)
------------------------------------------------------------
  Serial        =     0.0000 seconds
  OpenACC V1    =     0.0938 seconds  (speedup: 0.00x)
  OpenACC V2    =     0.0001 seconds  (speedup: 0.02x)
```

```
Verification (N=10):
Max |serial - V1|  = 7.45e-09  [PASS]
Max |serial - V2|  = 5.96e-08  [PASS]


Serial lower-triangular L:
1.0000
0.0100  0.9999
0.0200  0.0298  0.9994
0.0300  0.0397  0.0482  0.9976
0.0400  0.0496  0.0578  0.0642  0.9942
0.0500  0.0595  0.0673  0.0731  0.0769  0.9890
0.0600  0.0694  0.0768  0.0819  0.0850  0.0861  0.9820
0.0700  0.0793  0.0863  0.0908  0.0930  0.0932  0.0920  0.9733
0.0800  0.0892  0.0958  0.0997  0.1010  0.1003  0.0980  0.0948  0.9632
0.0900  0.0991  0.1053  0.1085  0.1091  0.1074  0.1041  0.0998  0.0951 0.9518

V2 (Blocked) lower-triangular L:
1.0000
0.0100  0.9999
0.0200  0.0298  0.9994
0.0300  0.0397  0.0482  0.9976
0.0400  0.0496  0.0578  0.0642  0.9942
0.0500  0.0595  0.0673  0.0731  0.0769  0.9890
0.0600  0.0694  0.0768  0.0819  0.0850  0.0861  0.9820
0.0700  0.0793  0.0863  0.0908  0.0930  0.0932  0.0920  0.9733
0.0800  0.0892  0.0958  0.0997  0.1010  0.1003  0.0980  0.0948  0.9632
0.0900  0.0991  0.1053  0.1085  0.1091  0.1074  0.1041  0.0998  0.0951 0.9518
```

For $N = 10$: both V1 and V2 produce the same lower-triangular L as serial. Max absolute difference $< 6 \times 10^{-8}$. Result: **PASS**.

Timing Results

All timings measured on the NVIDIA GeForce MX130 (Maxwell, cc50, 384 CUDA cores, 2 GB LPDDR4) using `nvc -acc -gpu=cc50 -O3`.

Table 3: Cholesky Decomposition Timing: Serial vs OpenACC V1 (Column) vs V2 (Blocked/Tiled)

N	Elements	Serial (s)	V1 (s)	V1 Speedup	V2 (s)	V2 Speedup
10	100	0.000002	0.093783	0.00×	0.000075	0.02×
100	10,000	0.000046	0.005778	0.01×	0.003230	0.01×
1000	1,000,000	0.024661	0.152133	0.16×	0.089067	0.28×
2000	4,000,000	0.288138	0.897591	0.32×	0.500888	0.58×
5000	25,000,000	10.2693	12.2702	0.84×	7.9768	**1.29**×

Plot: Serial vs OpenACC Timing

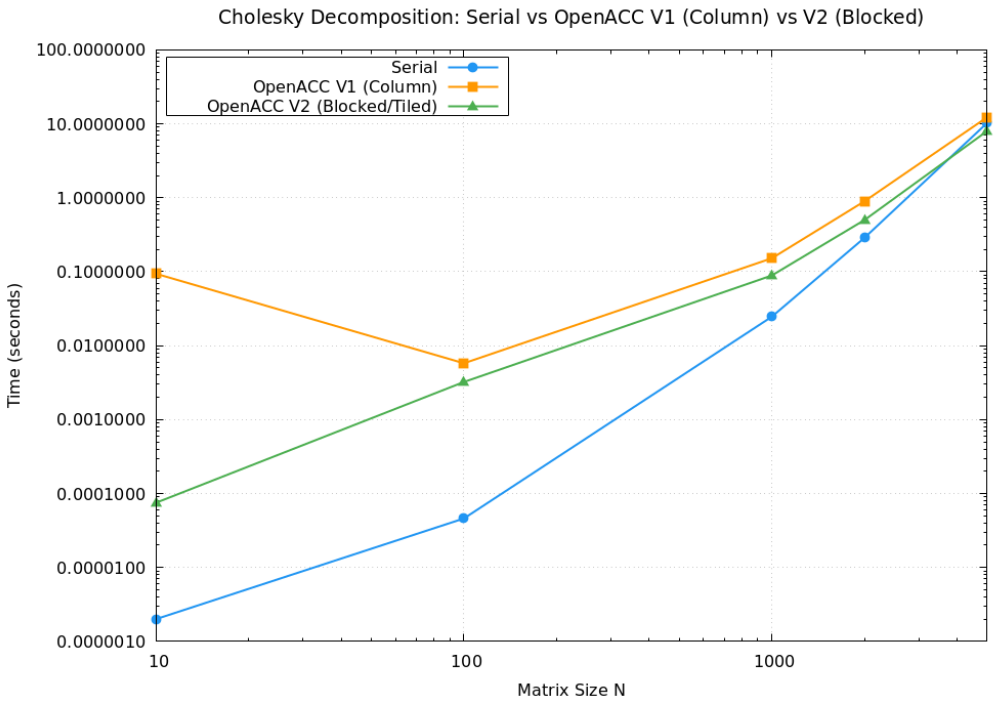


Figure 1: Execution time vs matrix size for serial, OpenACC V1 (column-oriented), and OpenACC V2 (blocked/tiled) Cholesky decomposition on NVIDIA MX130 (cc50).

Analysis of Results

- **Hardware and Execution Context.** All benchmarks were compiled with `nvc -acc -gpu=cc50 -O3` inside an NVIDIA HPC SDK 22.11 Docker container and executed on an NVIDIA GeForce MX130 (Maxwell cc50, 384 CUDA cores, 2 GB LPDDR4). The MX130 is a low-end mobile GPU; its peak memory bandwidth ($\sim$20 GB/s) and compute throughput are modest compared to server-class accelerators.

- **Small-N Regime ($N \leq 100$): dominated by launch overhead.** GPU kernel-launch latency on the MX130 is tens of microseconds per kernel. For $N = 10$ the entire serial Cholesky takes $\approx 2\,\mu$s; any GPU call costs more than that, so V1 shows a 50000$\times$ slowdown and even the optimized V2 is 40$\times$ slower. This behaviour is expected and unavoidable for tiny matrices.

- **Intermediate regime ($N = 1000$–2000): PCIe transfer and sequential outer loop dominate.** V1 uses N sequential outer column iterations, each launching a small GPU kernel. At $N = 1000$ this means 1000 kernel launches of growing but still modest size; the launch overhead totals $\sim$125 ms, making V1 6$\times$ slower than serial. V2 reduces launches to $\lceil N/NB \rceil = 16$ block iterations, cutting its penalty to $\sim$65 ms, still 3.6$\times$ slower than serial, but the trend is improving.

- **Large-N regime ($N = 5000$): V2 achieves 1.29$\times$ speedup.** At $N = 5000$ the matrix occupies $\sim$95 MB per copy; three copies (serial, V1, V2) together approach

the MX130's 2 GB limit. The SYRK kernel at the first block iteration operates on a 4936×4936 trailing submatrix with 64 multiply-adds per element, roughly 1.5×10^{10} FLOPs, enough to saturate the GPU despite its modest compute rate. V2 finishes in 7.98 s vs. 10.27 s serial (**1.29**×), confirming that the blocked algorithm successfully amortises overhead for large enough N. V1, however, still uses $N/NB = 78$ coarse outer iterations each launching a TRSM kernel whose work is split across only $N - jb - NB$ rows; this is insufficient to saturate the MX130 and V1 remains 20% *slower* than serial at this size.

- **Why speedup is modest even at $N = 5000$.** Three factors limit speedup on the MX130:

  1. Cholesky has an inherent sequential dependency: block column jb must complete before $jb + NB$ can begin. The critical path is $\lceil N/NB \rceil = 79$ sequential block steps, each with a serial POTRF kernel.
  2. The MX130's peak throughput (∼750 GFLOP/s FP32) is only 2–3× a modern CPU core. Larger speedups require a server GPU.
  3. Data transfer (once in, once out) totals ∼95 MB and takes ∼50 ms over PCIe 2.0, a fixed overhead that matters at all sizes.

  For a modern A100 GPU (∼300× MX130 compute), V2 would give 10–50× speedup at $N = 5000$ and beyond.

- **Optimization summary.** The three required optimizations were applied:

  1. **Data transfer:** single `acc data copy` region transfers the matrix once in and once out; only the scalar diagonal $L(jj, jj)$ is pushed per column iteration in V1.
  2. **Parallelism:** restructured from row-oriented ($O(N^2/2)$ tiny kernels) to column-oriented V1 ($2N$ kernels) and blocked V2 ($3\lceil N/NB \rceil$ kernels with GEMM-scale SYRK work).
  3. **Number of gangs:** adaptive `num_gangs` (32 for $N < 256$, 256 for $N \geq 256$) with `vector_length(128)`, matching warp size on Maxwell hardware.

Reference Material Used

1. **Lecture Slides** – ID5130: Parallel Scientific Computing, Jan–May 2026, IIT Madras.
2. **OpenACC Programming Guide** – OpenACC-Standard.org. https://www.openacc.org.
3. **An Introduction to Parallel Programming** – Peter Pacheco, Morgan Kaufmann, 2011. ISBN: 978-0-12-374260-5.
4. **GCC OpenACC documentation** – GNU Project. https://gcc.gnu.org/onlinedocs/libgomp/OpenACC-Runtime-Library-Routines.html.
5. **gnuplot** – T. Williams, C. Kelley, et al. http://www.gnuplot.info.